Day 3 React Curriculum Plan

Core Focus	Topics & Concepts Covered	Practical Lab / Deliverables
Lifecycle Hooks & Side Effects	• Introduction to the useEffect Hook • Dependency array rules ([], [dependencies], no array) • Side-effect management and cleanups • Fetch API inside useEffect with async/await • Handling loading skeletons and error states	**React Exercise 4: Posts Viewer (Part 1)** • Fetching 100 posts from JSONPlaceholder on mount • Managing component fetch state
API Integration & Data Rendering	• Consuming RESTful endpoints into local state • Tabular data rendering via .map() and key assignment • Defensive data checks and network error handling • End-to-end component verification	**React Exercise 4: Posts Viewer (Part 2)** • Building the Posts Table (Post ID, Title, Description) • Playwright automated test suite execution (7 cases)
Client-Side Routing with React Router	• Single Page Application (SPA) architecture • react-router-dom configuration: BrowserRouter, Routes, Route • Client-side navigation with <Link> (zero page reloads) • Dynamic multi-view layouts	**React Exercise 5: Student Portal** • 4-page navigation (Dashboard, Profile, Attendance, Marks) • Playwright automated test suite execution (7 cases)

NGIT/KMEC

Lifecycle Hooks (useEffect), Side Effects & API Integration

1. Theory & Core Concepts

A. What are Side Effects in React? In React, a component's primary job is to calculate and return JSX based on current props and state. Any operation that interacts with the outside world beyond this pure render cycle is considered a **Side Effect**.

- Fetching data from external REST APIs
- Setting up timers or intervals (setInterval, setTimeout)
- Manually modifying the browser DOM (e.g., updating document.title)
- Setting up subscriptions or event listeners on the window object

React provides the useEffect Hook to synchronize functional components with external systems without blocking the initial UI render.

B. Anatomy and Execution Flow of useEffect The useEffect Hook takes two arguments: a callback function containing the side-effect logic, and an optional dependency array.

```
useEffect(() => {
  // Side-effect logic executed here

  return () => {
    // Optional cleanup logic
  };
}, [dependencies]);
```

C. Dependency Array Execution Rules How and when useEffect triggers depends entirely on the second argument:

Dependency Configuration	When It Executes	Typical Use Case
No Dependency Array useEffect(() => { ... })	Runs after the initial render **and after every re-render**.	Logging renders, general UI synchronization.
Empty Dependency Array ([]) useEffect(() => { ... }, [])	Runs **strictly once** when the component first mounts to the DOM.	Initial API calls, fetching static configuration data.
With Dependencies ([prop, state]) useEffect(() => { ... }, [query])	Runs on mount and **whenever any specified dependency changes**.	Auto-complete searches, pagination, fetching user details on ID change.

D. Managing Asynchronous API Calls Inside useEffect The callback passed to useEffect cannot be directly declared as an async function (e.g., useEffect(async () => ...) is invalid because effects must return either nothing or a cleanup function). Instead, declare an internal async function inside the effect and invoke it immediately:

```
useEffect(() => {
  const loadData = async () => {
    try {
```

NGITKMEC

```
      const response = await
fetch("https://jsonplaceholder.typicode.com/posts"); //[cite: 1, 5]
      if (!response.ok) throw new Error(`HTTP Error: ${response.status}`);
      const data = await response.json();
      setPosts(data);
    } catch (err) {
      console.error(err.message);
    }
  };

  loadData();
}, []); // Empty array ensures data is fetched only once on startup[cite: 1, 3]
```

E. Lifecycle Cleanup Functions If an effect establishes a continuous process (such as a timer or socket subscription), returning a cleanup function prevents memory leaks by canceling the operation when the component unmounts or before the effect re-runs:

```
useEffect(() => {
  const timer = setInterval(() => {
    console.log("Tick");
  }, 1000);

  // Cleanup runs on unmount
  return () => clearInterval(timer); //[cite: 3]
}, []);
```

2. Hands-On Lab: React Exercise 4 (Posts Viewer - Part 1)

Problem Statement: Create a React application that retrieves post data from https://jsonplaceholder.typicode.com/posts using the Fetch API. Fetch the data inside the useEffect Hook and store it using useState. Display all posts in a table with **Post ID**, **Title**, and **Description** columns. Ensure the posts are loaded automatically when the application starts.

Project Directory Structure:

```
posts-viewer-app/
├── src/
│   ├── components/
│   │   └── PostsViewer.jsx    # Stateful API Fetching & Table Component
│   ├── App.jsx                # Root Application Component
│   ├── App.css                # Layout & Table Styling
│   └── main.jsx               # React DOM Entry Point
├── index.html
└── package.json
```

3. Step-by-Step Code Implementation

Step 1: Create the Posts Viewer Component (src/components/PostsViewer.jsx)

```
// src/components/PostsViewer.jsx
import React, { useState, useEffect } from 'react';

function PostsViewer() {
  // 1. State for storing fetched posts array
  const [posts, setPosts] = useState([]);

  // 2. State for loading and error tracking
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
```

NGITKMEC

```
  // 3. Lifecycle Hook: Trigger fetch on mount
  useEffect(() => {
    const fetchPosts = async () => {
      try {
        const response = await
fetch('https://jsonplaceholder.typicode.com/posts');
        if (!response.ok) {
          throw new Error(`Failed to fetch posts: ${response.status}`);
        }
        const data = await response.json();
        setPosts(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };

    fetchPosts();
  }, []); // Empty array ensures single execution on mount

  return (
    <div className="posts-container">
      <h1 className="main-heading">Posts Viewer</h1>

      {loading && <p className="status-message">Loading posts...</p>}
      {error && <p className="status-error">Error: {error}</p>}

      {!loading && !error && (
        <table className="posts-table">
          <thead>
            <tr>
              <th>Post ID</th>
              <th>Title</th>
              <th>Description</th>
            </tr>
          </thead>
          <tbody>
            {posts.map((post) => (
              <tr key={post.id} className="post-row">
                <td className="post-id-cell">{post.id}</td>
                <td className="post-title-cell">{post.title}</td>
                <td className="post-body-cell">{post.body}</td>
              </tr>
            ))}
          </tbody>
        </table>
      )}
    </div>
  );
}

export default PostsViewer;
```

Step 2: Connect Component in Root (src/App.jsx)

NGIT/KMEC

```jsx
// src/App.jsx
import React from 'react';
import PostsViewer from './components/PostsViewer';
import './App.css';

function App() {
  return (
    <div className="app-root">
      <PostsViewer />
    </div>
  );
}

export default App;
```

Step 3: Add Styling (src/App.css)

```css
/* src/App.css */
.app-root {
  min-height: 100vh;
  background-color: #f8fafc;
  font-family: Arial, sans-serif;
  padding: 2rem 1rem;
}

.posts-container {
  max-width: 1000px;
  margin: 0 auto;
  background-color: #ffffff;
  border: 1px solid #e2e8f0;
  border-radius: 8px;
  padding: 2rem;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
}

.main-heading {
  text-align: center;
  color: #0f172a;
  margin-bottom: 1.5rem;
}

.status-message {
  text-align: center;
  color: #64748b;
  font-size: 1.1rem;
}

.status-error {
  text-align: center;
  color: #ef4444;
  font-weight: bold;
}

.posts-table {
  width: 100%;
  border-collapse: collapse;
  text-align: left;
```

NGIT/KMEC

```
  margin-top: 1rem;
}

.posts-table th,
.posts-table td {
  padding: 0.85rem 1rem;
  border-bottom: 1px solid #e2e8f0;
  vertical-align: top;
}

.posts-table th {
  background-color: #f1f5f9;
  color: #334155;
  font-weight: 600;
}

.post-id-cell {
  font-weight: bold;
  color: #2563eb;
  width: 80px;
}

.post-title-cell {
  font-weight: 600;
  color: #1e293b;
  width: 250px;
}

.post-body-cell {
  color: #475569;
  line-height: 1.4;
}
```

4. Execution Guidelines

1. Initialize project using Vite:

```
npm create vite@latest posts-viewer-app -- --template react
cd posts-viewer-app
npm install
```

2. Place PostsViewer.jsx in src/components/, update src/App.jsx, and replace src/App.css.
3. Start the dev server:

```
npm run dev
```

4. Verify in the browser (http://localhost:5173) that the table renders all 100 records automatically with Post ID, Title, and Description.

NGIT/KMEC

REST API Integration, Dynamic Tabular Rendering & Playwright Assertions

1. Theory & Core Concepts

A. Consuming RESTful APIs in React Components

- **Request Lifecycle:** Data ingestion typically requires tracking three distinct operational states: data (payload array/object), loading (boolean flag to show skeletons/spinners), and error (network or HTTP failure messages).

- **Defensive HTTP Parsing:** The Fetch API does not automatically reject Promises on standard HTTP error responses (e.g., 404 Not Found or 500 Server Error). Validate the response.ok status boolean before invoking .json() to catch response faults.

B. Tabular Data Rendering Patterns

- **Collection Mapping:** Ingested JSON arrays are projected into HTML <table> structures using .map().

- **Reconciliation Identity (**key**):** Assign a stable and unique property from the API payload (e.g., post.id) to each <tr> element to allow React's Virtual DOM diffing engine to track row mutations efficiently.

C. Automated UI Verification with Playwright

- **Asynchronous Assertions:** When testing API-driven tables, Playwright automatically waits for network resolutions, allowing direct assertions on table counts, column headers, and specific cell contents.

2. Hands-On Lab: React Exercise 4 (Posts Viewer - Part 2)

Problem Statement: Create a React application that retrieves post data from https://jsonplaceholder.typicode.com/posts using the Fetch API. Fetch the data inside the useEffect Hook and store it using useState. Display all posts in a table with **Post ID**, **Title**, and **Description** columns. Ensure the posts are loaded automatically when the application starts.

Project Directory Structure:

Plaintext

```
posts-viewer-app/
├── src/
│   ├── components/
│   │   └── PostsViewer.jsx    # Stateful Component Rendering Posts Table
│   ├── App.jsx              # Main Application Container
│   ├── App.css              # Clean Table & State Styles
│   └── main.jsx             # DOM Mounting Point
├── tests/
│   └── postsViewer.spec.js    # Automated Playwright Verification Suite
├── index.html
└── package.json
```

3. Step-by-Step Code Implementation

Step 1: Create Posts Component (src/components/PostsViewer.jsx**)**

NGTAMEC

JavaScript

```
// src/components/PostsViewer.jsx
import React, { useState, useEffect } from 'react';

function PostsViewer() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchPosts = async () => {
      try {
        const response = await fetch('https://jsonplaceholder.typicode.com/posts');
        if (!response.ok) {
          throw new Error(`HTTP Error: ${response.status}`);
        }
        const data = await response.json();
        setPosts(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };

    fetchPosts();
  }, []); // Run strictly once on mount

  return (
    <div className="posts-container">
      <h1 className="posts-heading">Posts Viewer</h1>

      {loading && <p className="loading-state">Loading posts...</p>}
      {error && <p className="error-state">Error: {error}</p>}

      {!loading && !error && (
        <table className="posts-table">
          <thead>
            <tr>
              <th>Post ID</th>
              <th>Title</th>
              <th>Description</th>
            </tr>
          </thead>
          <tbody>
            {posts.map((post) => (
              <tr key={post.id} className="post-row">
                <td className="post-id">{post.id}</td>
```

NGIT/KMEC

```
            <td className="post-title">{post.title}</td>
            <td className="post-desc">{post.body}</td>
          </tr>
        ))}
      </tbody>
    </table>
  )}
 </div>
 );
}

export default PostsViewer;
```

Step 2: Mount Component in Main App (src/App.jsx**)**

```
JavaScript
// src/App.jsx
import React from 'react';
import PostsViewer from './components/PostsViewer';
import './App.css';

function App() {
 return (
  <div className="app-container">
   <PostsViewer />
  </div>
 );
}

export default App;
```

Step 3: Component Styling (src/App.css**)**

```
CSS
/* src/App.css */
.app-container {
 display: flex;
 justify-content: center;
 padding: 2rem;
 background-color: #f8fafc;
 min-height: 100vh;
 font-family: Arial, sans-serif;
}

.posts-container {
 width: 100%;
 max-width: 960px;
 background: #ffffff;
 padding: 2rem;
 border-radius: 8px;
 border: 1px solid #e2e8f0;
```

NGIT/KMEC

```
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.05);
}

.posts-heading {
  text-align: center;
  color: #0f172a;
  margin-bottom: 1.5rem;
}

.loading-state, .error-state {
  text-align: center;
  font-size: 1.1rem;
  margin-top: 1rem;
}

.error-state {
  color: #ef4444;
}

.posts-table {
  width: 100%;
  border-collapse: collapse;
  text-align: left;
}

.posts-table th, .posts-table td {
  padding: 0.75rem 1rem;
  border-bottom: 1px solid #e2e8f0;
  vertical-align: top;
}

.posts-table th {
  background-color: #f1f5f9;
  color: #334155;
  font-weight: 600;
}

.post-id {
  font-weight: bold;
  color: #2563eb;
  width: 10%;
}

.post-title {
  font-weight: 600;
  color: #1e293b;
  width: 30%;
}

.post-desc {
```

NGIT/KMEC

```
  color: #64748b;
  width: 60%;
  line-height: 1.4;
}
```

4. Automated Playwright Verification Suite

File: tests/postsViewer.spec.js

JavaScript

```
// tests/postsViewer.spec.js
import { test, expect } from '@playwright/test';

test.describe('React Exercise 4: Posts Viewer Verification', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('http://localhost:5173');
  });

  // 1. Verify heading
  test('1. Verify heading', async ({ page }) => {
    const heading = page.locator('h1.posts-heading');
    await expect(heading).toHaveText('Posts Viewer');
  });

  // 2. Table visible
  test('2. Table visible', async ({ page }) => {
    const table = page.locator('table.posts-table');
    await expect(table).toBeVisible();
  });

  // 3. Three headers
  test('3. Three headers', async ({ page }) => {
    const headers = page.locator('table.posts-table th');
    await expect(headers).toHaveCount(3);
    await expect(headers.nth(0)).toHaveText('Post ID');
    await expect(headers.nth(1)).toHaveText('Title');
    await expect(headers.nth(2)).toHaveText('Description');
  });

  // 4. Posts loaded
  test('4. Posts loaded', async ({ page }) => {
    const rows = page.locator('tbody tr.post-row');
    await expect(rows.first()).toBeVisible();
  });

  // 5. First ID
  test('5. First ID', async ({ page }) => {
    const firstId = page.locator('.post-id').first();
    await expect(firstId).toHaveText('1');
  });
```

NCT/KMEC

```
  // 6. First title
  test('6. First title', async ({ page }) => {
    const firstTitle = page.locator('.post-title').first();
    await expect(firstTitle).not.toBeEmpty();
  });

  // 7. 100 rows displayed
  test('7. 100 rows displayed', async ({ page }) => {
    const rows = page.locator('tbody tr.post-row');
    await expect(rows).toHaveCount(100);
  });
});
```

Client-Side Routing with React Router (react-router-dom)

1. Theory & Core Concepts

A. Single Page Applications (SPA) vs. Multi-Page Applications (MPA)

- **Traditional MPAs:** Every link click requests a brand-new HTML document from the web server, causing screen flashing, loss of JavaScript runtime memory, and full-page reloads.
- **Modern SPAs:** The browser loads a single root HTML shell (index.html) once. Navigation is intercepted client-side via JavaScript, seamlessly mounting and unmounting view components while updating the URL in the browser address bar without reloads.

B. Core Components of react-router-dom

- **BrowserRouter:** The top-level router wrapper that connects your React application to the browser's HTML5 History API (pushState, replaceState).
- **Routes:** The container that evaluates all defined <Route> children and selectively renders the first matching path.
- **Route:** Defines the mapping between a specific URL path pattern and the component to mount (<Route element="{<Profile" path="/profile"/>} />).
- **<Link to="...">:** The navigation element that replaces standard <a href="..."> tags. It updates the URL and changes the active view without triggering a network fetch or browser refresh.

2. Hands-On Lab: React Exercise 5 (Student Portal)

Problem Statement: Develop a Student Portal using React Router. Create four pages: **Dashboard**, **Profile**, **Attendance**, and **Marks**. Provide navigation links that allow users to move between these pages without refreshing the browser. Configure routing using BrowserRouter, Routes, Route, and Link from react-router-dom.

Project Directory Structure:

```
student-portal-app/
├── src/
│   ├── pages/
│   │   ├── Dashboard.jsx     # Overview & Welcome View
│   │   ├── Profile.jsx       # Student Details View
│   │   ├── Attendance.jsx    # Monthly Attendance View
│   │   └── Marks.jsx         # Academic Scorecard View
│   ├── components/
│   │   └── Navbar.jsx        # Navigation Header with Links
│   ├── App.jsx               # Central Route Switchboard
│   ├── App.css               # Portal Layout & Styling
│   └── main.jsx              # Application Root (wrapped with BrowserRouter)
```

NOT KNEC

```
├── index.html
└── package.json
```

3. Step-by-Step Code Implementation

Step 1: Install react-router-dom

```
npm install react-router-dom
```

Step 2: Create the 4 Page Views (src/pages/)

- src/pages/Dashboard.jsx:

```
// src/pages/Dashboard.jsx
import React from 'react';

function Dashboard() {
  return (
    <div className="portal-card">
      <h2>Student Dashboard</h2>
      <p className="welcome-text">
        Welcome back, <strong>Aarav Sharma</strong>! Select a tab above to
manage your academic profile, view daily attendance, or review semester marks.
      </p>
    </div>
  );
}

export default Dashboard;
```

- src/pages/Profile.jsx:

```
// src/pages/Profile.jsx
import React from 'react';

function Profile() {
  return (
    <div className="portal-card">
      <h2>Student Profile</h2>
      <div className="profile-details">
        <p className="profile-name"><strong>Full Name:</strong> Aarav
Sharma</p>
        <p className="profile-roll"><strong>Roll Number:</strong>
21BD1A0501</p>
        <p className="profile-branch"><strong>Branch:</strong> Computer Science
& Engineering</p>
        <p className="profile-email"><strong>Email:</strong>
aarav.sharma@college.edu</p>
      </div>
    </div>
  );
}

export default Profile;
```

- src/pages/Attendance.jsx:

```
// src/pages/Attendance.jsx
import React from 'react';

function Attendance() {
```

NGIT/KMEC

```
  return (
    <div className="portal-card">
      <h2>Attendance Tracker</h2>
      <div className="attendance-metric">
        <p className="metric-score">88.5%</p>
        <p className="metric-label">Overall Semester Attendance</p>
      </div>
    </div>
  );
}

export default Attendance;
```

- src/pages/Marks.jsx:

```
// src/pages/Marks.jsx
import React from 'react';

function Marks() {
  const scoreData = [
    { code: 'CS501', subject: 'Web Technologies', score: 92 },
    { code: 'CS502', subject: 'Database Management Systems', score: 85 },
    { code: 'CS503', subject: 'Software Engineering', score: 78 }
  ];

  return (
    <div className="portal-card">
      <h2>Academic Marks</h2>
      <table className="marks-table">
        <thead>
          <tr>
            <th>Subject Code</th>
            <th>Subject Name</th>
            <th>Marks Obtained</th>
          </tr>
        </thead>
        <tbody>
          {scoreData.map((item) => (
            <tr key={item.code} className="marks-row">
              <td>{item.code}</td>
              <td>{item.subject}</td>
              <td className="score-cell">{item.score} / 100</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default Marks;
```

Step 3: Create Navigation Component (src/components/Navbar.jsx)

```
// src/components/Navbar.jsx
import React from 'react';
import { Link } from 'react-router-dom';
```

NGIT/KMEC

```
function Navbar() {
  return (
    <nav className="portal-navbar">
      <Link to="/" className="nav-link nav-dashboard">Dashboard</Link>
      <Link to="/profile" className="nav-link nav-profile">Profile</Link>
      <Link to="/attendance" className="nav-link nav-
attendance">Attendance</Link>
      <Link to="/marks" className="nav-link nav-marks">Marks</Link>
    </nav>
  );
}

export default Navbar;
```

Step 4: Configure the Router Tree (src/App.jsx)

```
// src/App.jsx
import React from 'react';
import { Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';
import Dashboard from './pages/Dashboard';
import Profile from './pages/Profile';
import Attendance from './pages/Attendance';
import Marks from './pages/Marks';
import './App.css';

function App() {
  return (
    <div className="portal-container">
      <h1 className="page-heading">Student Portal</h1>
      <Navbar />

      <div className="portal-content">
        <Routes>
          <Route path="/" element={<Dashboard />} />
          <Route path="/profile" element={<Profile />} />
          <Route path="/attendance" element={<Attendance />} />
          <Route path="/marks" element={<Marks />} />
        </Routes>
      </div>
    </div>
  );
}

export default App;
```

Step 5: Wrap Root in BrowserRouter (src/main.jsx)

```
// src/main.jsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
```

NGIT/KMEC

```
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

Step 6: Add Layout & Navigation Styles (src/App.css)

```
/* src/App.css */
body {
  margin: 0;
  background-color: #f1f5f9;
  font-family: Arial, sans-serif;
}

.portal-container {
  max-width: 800px;
  margin: 2rem auto;
  padding: 1.5rem;
}

.page-heading {
  text-align: center;
  color: #0f172a;
  margin-bottom: 1.5rem;
}

.portal-navbar {
  display: flex;
  justify-content: center;
  gap: 1rem;
  background-color: #ffffff;
  padding: 0.75rem;
  border-radius: 8px;
  border: 1px solid #cbd5e1;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.05);
}

.nav-link {
  text-decoration: none;
  color: #2563eb;
  font-weight: 600;
  padding: 0.5rem 1rem;
  border-radius: 6px;
  transition: background-color 0.2s ease;
}

.nav-link:hover {
  background-color: #eff6ff;
}

.portal-content {
  margin-top: 1.5rem;
}

.portal-card {
  background-color: #ffffff;
```

NGIT/KMEC

```
  padding: 1.5rem 2rem;
  border-radius: 8px;
  border: 1px solid #e2e8f0;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.05);
}

.portal-card h2 {
  color: #1e293b;
  margin-top: 0;
  border-bottom: 2px solid #e2e8f0;
  padding-bottom: 0.5rem;
}

.profile-details p {
  margin: 0.6rem 0;
  color: #334155;
  font-size: 1rem;
}

.attendance-metric {
  text-align: center;
  padding: 1.5rem 0;
}

.metric-score {
  font-size: 2.5rem;
  font-weight: bold;
  color: #16a34a;
  margin: 0;
}

.metric-label {
  color: #64748b;
  margin-top: 0.25rem;
}

.marks-table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 1rem;
}

.marks-table th,
.marks-table td {
  padding: 0.75rem 1rem;
  border-bottom: 1px solid #e2e8f0;
  text-align: left;
}

.marks-table th {
  background-color: #f8fafc;
  color: #475569;
}

.score-cell {
  font-weight: bold;
```

NGIT/KMEC

```
  color: #0f172a;
}
```

4. Verification Checkpoints

- Visiting http://localhost:5173/ renders the **Dashboard** by default.
- Clicking **Profile**, **Attendance**, or **Marks** links updates the active component immediately with zero page reloads or screen flashes.
- The browser back/forward buttons seamlessly track and navigate previous page states.

Would you like to proceed with the curriculum plan and module breakdown for **Day 4: Global Context API (useContext), Express.js REST APIs & Full-Stack Architecture**?

User prompt: give me the theory part along with syntaxes of day 3

Response: **Day 3: Complete Theoretical Guide & Core Syntaxes Theme:** Lifecycle Hooks, REST API Integration & Single Page Routing (react-router-dom)

1. React Side Effects & The useEffect Hook

A. Concept of Side Effects In React, a component's primary rendering logic is pure—it calculates and returns JSX based on its current props and state. A **Side Effect** is any operation that reaches outside the functional scope of that render cycle to interact with external systems:

- Fetching data from a REST API backend.
- Setting up manual subscriptions or timers (setInterval, setTimeout).
- Directly manipulating browser APIs or the DOM (e.g., modifying document.title).
- Attaching and detaching global event listeners on the window object.

React provides the built-in useEffect Hook to encapsulate, execute, and clean up these side effects safely without blocking UI rendering.

B. General Syntax of useEffect

```
import { useEffect } from 'react';

useEffect(() => {
  // 1. Side-effect logic executed after render

  return () => {
    // 2. Optional cleanup function (runs on unmount or before effect re-runs)
  };
}, [/* 3. Dependency Array */]);
```

2. The Dependency Array Mechanics

The execution lifecycle of useEffect is governed by the values passed in its second argument (the dependency array):

Dependency Configuration	**Syntax**	**Execution Timing**	**Use Case**
No Array	useEffect(() => { ... });	Runs after initial mount **and after every re-render**.	General UI synchronization, logging render

NGIT/KMEC

Dependency Configuration	Syntax	Execution Timing	Use Case
			metrics.
Empty Array ([])	useEffect(() => { ... }, []);	Runs **strictly once** when the component first mounts.	Initial REST API fetches, loading global config.
With Dependencies	useEffect(() => { ... }, [id]);	Runs on mount and **whenever any specified dependency updates**.	Search auto-completes, pagination, dynamic route-based fetching.

3. REST API Integration & Cleanups Inside useEffect

A. Asynchronous Data Fetching Pattern The callback function passed to useEffect cannot be declared async directly because React expects effects to return either a cleanup function or undefined. Instead, declare an internal asynchronous function inside the effect and call it immediately:

```
import React, { useState, useEffect } from 'react';

function DataFetcher() {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    // Define internal async worker
    const fetchData = async () => {
      try {
        const response = await
fetch('https://jsonplaceholder.typicode.com/posts');
        if (!response.ok) {
          throw new Error(`HTTP Error: ${response.status}`);
        }
        const result = await response.json();
        setData(result);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };

    fetchData(); // Invoke immediately
  }, []); // [] ensures API is called only once when the component mounts
}
```

B. Effect Cleanup Functions (Preventing Memory Leaks) When an effect establishes a persistent

NGIT/KMEC

subscription, interval, or event listener, it must return a cleanup function to release system resources when the component unmounts:

```
useEffect(() => {
  const timer = setInterval(() => {
    console.log('Periodic heartbeat tick...');
  }, 1000);

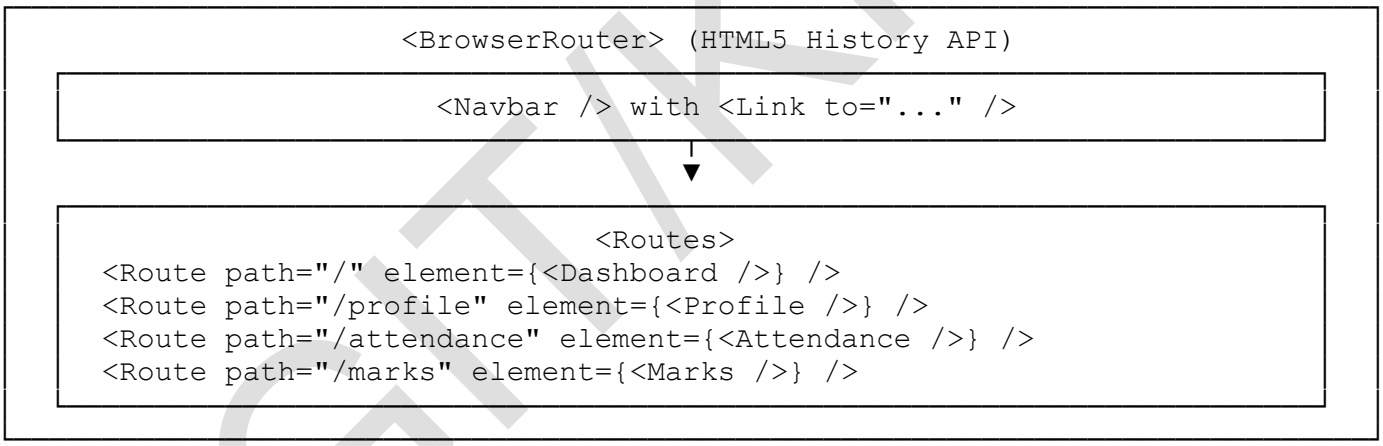
  // Cleanup function executed on component unmount
  return () => {
    clearInterval(timer);
  };
}, []);
```

4. Client-Side Routing & Single Page Applications (SPA)

A. SPA vs. Traditional Multi-Page Applications (MPA)

- **Traditional MPA:** Clicking a navigation link sends a new HTTP GET request to the server, resulting in a full browser reload, screen flashing, and reset of in-memory JavaScript state.
- **Single Page Application (SPA):** The browser loads one root HTML document (index.html) once. Navigation is handled entirely on the client side via JavaScript, mounting and unmounting page components dynamically while updating the URL via the HTML5 History API without reloading the page.

B. The Core Building Blocks of react-router-dom



- **BrowserRouter:** Top-level provider component that wraps the entire React app to enable client-side routing capabilities.
- **Routes:** Parent container component that inspects the current URL and renders the first matching child <Route>.
- **Route:** Configures an individual route path and pairs it with a corresponding component (path="/..." element={<Component/>}).
- **<Link to="...">:** Replaces standard HTML <a href="..."> anchors to prevent full-page browser refreshes and manage instant component switching.

5. react-router-dom Syntaxes

A. Mounting the Router Provider (src/main.jsx)

```
// src/main.jsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
```

NGIT/MEC

```
import App from './App';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

B. Navigation Component with <Link> (src/components/Navbar.jsx)

```
// src/components/Navbar.jsx
import React from 'react';
import { Link } from 'react-router-dom';

function Navbar() {
  return (
    <nav className="navbar">
      {/* 'to' prop replaces 'href' to avoid full page reloads */}
      <Link to="/">Dashboard</Link>
      <Link to="/profile">Profile</Link>
      <Link to="/attendance">Attendance</Link>
      <Link to="/marks">Marks</Link>
    </nav>
  );
}

export default Navbar;
```

C. Central Route Switchboard Configuration (src/App.jsx)

```
// src/App.jsx
import React from 'react';
import { Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';
import Dashboard from './pages/Dashboard';
import Profile from './pages/Profile';
import Attendance from './pages/Attendance';
import Marks from './pages/Marks';

function App() {
  return (
    <div className="app-container">
      <Navbar />

      {/* Route matching container */}
      <main className="main-content">
        <Routes>
          <Route path="/" element={<Dashboard />} />
          <Route path="/profile" element={<Profile />} />
          <Route path="/attendance" element={<Attendance />} />
          <Route path="/marks" element={<Marks />} />
        </Routes>
      </main>
    </div>
  );
```

NGIT/KMEC

```
}

export default App;
```

NGIT/KMEC